

HDOC – Day 20

C Programming Assignment

Name - Arushi Gusain

Sap id - 590033489

Batch 15

Date - 13/09/2026

Application of Iterative and Decision-Making Constructs to Shapes and Physics using Modular Programming in C

Deliverables included: algorithms, test-case table, modular source code, compilation command, execution evidence, and test results.

Constants: $\text{PI} = 3.1416$, $\text{GRAVITY} = 9.8 \text{ m/s}^2$, $\text{TOLERANCE} = 0.000001$. All dimensions/elevation are in metres and densities in kg/m^3 .

1. Problem Analysis and Module Design

Module	Responsibility
main.c	Menu, input validation, dispatch with switch-case, result display.
geometry.c	Surface area, volume, base-area and cone slant-height calculations.
physics.c	Mass, weight, potential energy, pressure, buoyant force and buoyancy behavior.
math_utils.c	Square root using Newton-Raphson iteration without math.h.
solid.h	Header guard, constants, symbolic choices and external function declarations.

2. Algorithms

Main menu

1. Start. 2. Use a do-while loop. 3. Display choices 1-10. 4. Read choice. 5. Use switch-case to call the required operation. 6. Validate every input immediately; repeat invalid input. 7. Display the result with SI unit. 8. Continue until choice 10. 9. Stop.

Surface area

Read and validate solid type. Read positive dimensions. Sphere: $4\pi r^2$. Cylinder: $2\pi r(r+h)$. Cone: compute slant height using $\text{calculate_square_root}(r^2+h^2)$, then $\pi r(r+\text{slant})$. Cuboid: $2(lb+lh+bh)$. Return/display area.

Volume

Read solid and positive dimensions. Sphere: $(4/3)\pi r^3$. Cylinder: $\pi r^2 h$. Cone: $(1/3)\pi r^2 h$. Cuboid: $l b h$. Return/display volume.

Base area

Read solid. If sphere, display “not defined” and return. For cylinder/cone read positive radius and compute πr^2 . For cuboid read positive length and breadth and compute $l b$.

Mass

Read positive density. Read solid and valid dimensions. Obtain volume using the corresponding geometry function. Compute mass = density*volume. Return mass.

Weight

Read positive solid density. Call `calculate_mass()`. Compute weight = mass*GRAVITY. Return weight.

Gravitational potential energy

Read elevation; reject negative values but allow zero. Read density and obtain mass using `calculate_mass()`. Compute PE = mass*GRAVITY*elevation. Return PE. Pressure on base Read positive density. Read solid and dimensions. Reject sphere. Obtain volume and contact area. Compute pressure=(density*volume*GRAVITY)/contact_area. Return pressure.

Buoyant force

Read positive fluid density. Read solid and dimensions. Obtain solid

volume. Compute buoyant force=fluid_density*volume*GRAVITY. Return force.

Buoyancy behavior

Read positive solid density and fluid density. If solid_density > fluid_density return SINKS (1); if less return FLOATS (-1); otherwise return SUSPENDED (0).

Newton-Raphson square root

If number<0 return -1; if number=0 return 0. Set old_value=number. In a do-while loop compute new_value=0.5*(old_value+number/old_value), difference=new_value-old_value, make difference positive if negative, set old_value = new_value. Repeat while difference>=TOLERANCE. Return new_value.

3. Test-Case Table

ID	Operation/Input	Type	Expected Result
T1	Surface area: sphere, r=2	Valid	50.2656 m2
T2	Volume: cylinder, r=2, h=5	Valid	62.8320 m3
T3	Surface area: cone, r=3, h=4	Valid	75.3984 m2 (slant=5)
T4	Volume: cuboid,l=2, b=3, h=4	Valid	24.0000 m3
T5	Base area: cylinder, r=2	Valid	12.5664 m2
T6	Base area: sphere	Invalid operation	Message: base area not defined for sphere
T7	Dimension r=0, then r=1	Boundary/invalid	0 rejected; 1 accepted
T8	Dimension h=-2, then h=2	Invalid	-2 rejected; 2 accepted
T9	Potential energy elevation=0	Boundary	0 J for any valid mass
T10	Mass: sphere r=1, density=1000	Valid	4188.8000 kg
T11	Pressure: cylinder r=1,h=2,density=500	Valid	9800.0000 Pa
T12	Buoyant force: cuboid 1x2x3, fluid density=1000	Valid	58800.0000 N

T13	Buoyancy: solid=7800, fluid=1000	Valid	SINKS
T14	Buoyancy: solid=900, fluid=1000	Valid	FLOATS
T15	Buoyancy: solid=1000, fluid=1000	Boundary	SUSPENDED
T16	Solid choice=5, then 2	Invalid	5 rejected; cylinder accepted
T17	Menu choice=0 or 11	Invalid	Invalid menu choice; menu repeats
T18	Density=0, then 1000	Boundary/invalid	0 rejected; 1000 accepted
T19	Square root input=0	Boundary	0
T20	Square root input=-4	Invalid mathematical input	-1 returned by utility function

4. Complete Modular C Program

4.1 solid.h

```

#ifndef SOLID_H
#define SOLID_H
#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001
#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4
#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

double sphere_area(double radius);

double sphere_volume(double radius);

double cylinder_area(double radius, double height);

```

```

double cylinder_volume(double radius, double height);
double cone_area(double radius, double height);
double cone_volume(double radius, double height);
double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);
double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);
double calculate_square_root(double number);
double calculate_mass(double density);
double calculate_weight(void);
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);
#endif

```

4.2 geometry.c

```

#include "solid.h"

double sphere_area(double radius) {
    return 4 * PI * radius * radius;
}

double sphere_volume(double radius) {
    return (4.0 / 3.0) * PI * radius * radius * radius;
}

double cylinder_area(double radius, double height) {
    return 2 * PI * radius * (radius + height);
}

```

```

}

double cylinder_volume(double radius, double height) {
return PI * radius * radius * height;
}

double cone_area(double radius, double height) {
double slant_height = calculate_square_root(radius * radius + height *
height);
return PI * radius * (radius + slant_height);
}

double cone_volume(double radius, double height) {
return (1.0 / 3.0) * PI * radius * radius * height;
}

double cuboid_area(double length, double breadth, double height) {
return 2 * (length * breadth + breadth * height + length * height);
}

double cuboid_volume(double length, double breadth, double height) {
return length * breadth * height;
}

double circular_base_area(double radius) {
return PI * radius * radius;
}

double cuboid_base_area(double length, double breadth) {
return length * breadth;
}

```

4.3 math_utils.c

```
#include "solid.h"

double calculate_square_root(double number) {
    double old_value, new_value, difference;

    if (number < 0) return -1;
    if (number == 0) return 0;

    old_value = number;

    do {
        new_value = 0.5 * (old_value + number / old_value);
        difference = new_value - old_value;
        if (difference < 0) difference = -difference;
        old_value = new_value;
    } while (difference >= TOLERANCE);

    return new_value;
}
```

4.4 physics.c

```
#include <stdio.h>

#include "solid.h"

static int solid_type;

static double r, h, l, b;

static int read_solid_dimensions(void) {
    printf("Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): ");
    scanf("%d", &solid_type);

    if (solid_type < SPHERE || solid_type > CUBOID) return 0;

    if (solid_type == SPHERE) {
```

```

printf("Enter radius (m): "); scanf("%lf", &r);

return r > 0;

}

if (solid_type == CYLINDER || solid_type == CONE) {
printf("Enter radius and height (m): "); scanf("%lf %lf", &r, &h);

return r > 0 && h > 0;

}

printf("Enter length, breadth and height (m): "); scanf("%lf %lf %lf", &l, &b,
&h);

return l > 0 && b > 0 && h > 0;

}

static double selected_volume(void) {
if (solid_type == SPHERE) return sphere_volume(r);
if (solid_type == CYLINDER) return cylinder_volume(r, h);
if (solid_type == CONE) return cone_volume(r, h);
return cuboid_volume(l, b, h);
}

double calculate_mass(double density) {
if (!read_solid_dimensions()) return -1;
return density * selected_volume();
}

double calculate_weight(void) {
double density, mass;

printf("Enter solid density (kg/m^3): "); scanf("%lf", &density);

if (density <= 0) return -1;

```

```

mass = calculate_mass(density);
if (mass < 0) return -1;
return mass * GRAVITY;
}

double potential_energy(double elevation) {
double density, mass;
printf("Enter solid density (kg/m^3): "); scanf("%lf", &density);
if (density <= 0) return -1;
mass = calculate_mass(density);
if (mass < 0) return -1;
return mass * GRAVITY * elevation;
}

double calculate_pressure(double density) {
double volume, contact_area;
if (!read_solid_dimensions()) return -1;
if (solid_type == SPHERE) return -2;
volume = selected_volume();
if (solid_type == CUBOID) contact_area = cuboid_base_area(l, b);
else contact_area = circular_base_area(r);
return (density * volume * GRAVITY) / contact_area;
}

double buoyant_force(double fluid_density) {
if (!read_solid_dimensions()) return -1;
return fluid_density * selected_volume() * GRAVITY;
}

```

```
int determine_buoyancy(double solid_density, double fluid_density) {  
    if (solid_density > fluid_density) return SINKS;  
    if (solid_density < fluid_density) return FLOATS;  
    return SUSPENDED;  
}
```

4.5 main.c

```
#include <stdio.h>  
  
#include "solid.h"  
  
static int get_solid(void) {  
    int s;  
    do {  
        printf("Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): ");  
        scanf("%d", &s);  
        if (s < 1 || s > 4) printf("Invalid solid choice. Try again.\n");  
    } while (s < 1 || s > 4);  
    return s;  
}  
  
static double positive(const char *prompt) {  
    double x;  
    do {  
        printf("%s", prompt); scanf("%lf", &x);  
        if (x <= 0) printf("Value must be greater than zero. Try again.\n");  
    } while (x <= 0);  
    return x;  
}
```

```

int main(void) {
    int choice, solid, state;
    double r, h, l, b, density, fluid_density, elevation, result;
    do {
        printf("\n=== SOLID GEOMETRY AND PHYSICS MENU ===\n");
        printf("1. Surface area\n2. Volume\n3. Base area\n4. Mass\n5. Weight\n");
        printf("6. Gravitational potential energy\n7. Pressure on base\n");
        printf("8. Buoyant force\n9. Buoyancy behavior\n10. Exit\nChoice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                solid = get_solid();
                if (solid == SPHERE) { r=positive("Radius (m): "); result=sphere_area(r); }
                else if (solid == CYLINDER) { r=positive("Radius (m): "); h=positive("Height (m): "); result=cylinder_area(r,h); }
                else if (solid == CONE) { r=positive("Radius (m): "); h=positive("Height (m): "); result=cone_area(r,h); }
                else { l=positive("Length (m): "); b=positive("Breadth (m): "); h=positive("Height (m): "); result=cuboid_area(l,b,h); }
                printf("Surface area = %.4f m^2\n", result); break;
            case 2:
                solid = get_solid();
                if (solid == SPHERE) { r=positive("Radius (m): "); result=sphere_volume(r); }
                else if (solid == CYLINDER) { r=positive("Radius (m): "); h=positive("Height (m): "); result=cylinder_volume(r,h); }

```

```
else if (solid == CONE) { r=positive("Radius (m): "); h=positive("Height (m): "); result=cone_volume(r,h); }
```

```
else { l=positive("Length (m): "); b=positive("Breadth (m): "); h=positive("Height (m): "); result=cuboid_volume(l,b,h); }
```

```
printf("Volume = %.4f m^3\n", result); break;
```

```
case 3:
```

```
solid = get_solid();
```

```
if (solid == SPHERE) { printf("Base area is not defined for a sphere.\n"); break; }
```

```
if (solid == CYLINDER || solid == CONE) { r=positive("Radius (m): "); result=circular_base_area(r); }
```

```
else { l=positive("Length (m): "); b=positive("Breadth (m): "); result=cuboid_base_area(l,b); }
```

```
printf("Base area = %.4f m^2\n", result); break;
```

```
case 4:
```

```
density=positive("Solid density (kg/m^3): "); result=calculate_mass(density);
```

```
if (result < 0) printf("Invalid solid/dimensions.\n"); else printf("Mass = %.4f kg\n",result); break;
```

```
case 5:
```

```
result=calculate_weight();
```

```
if (result < 0) printf("Invalid input.\n"); else printf("Weight = %.4f N\n",result); break;
```

```
case 6:
```

```
do { printf("Elevation (m, >= 0): "); scanf("%lf",&elevation); if(elevation<0) printf("Elevation cannot be negative.\n"); } while(elevation<0);
```

```
result=potential_energy(elevation);
```

```
if (result < 0) printf("Invalid input.\n"); else printf("Potential energy = %.4f
J\n",result); break;
```

```
case 7:
```

```
density=positive("Solid density (kg/m^3): ");
```

```
result=calculate_pressure(density);
```

```
if (result == -2) printf("Pressure on base is not defined for a sphere.\n");
```

```
else if (result < 0) printf("Invalid input.\n"); else printf("Pressure = %.4f
Pa\n",result); break;
```

```
case 8:
```

```
fluid_density=positive("Fluid density (kg/m^3): ");
```

```
result=buoyant_force(fluid_density);
```

```
if (result < 0) printf("Invalid input.\n"); else printf("Buoyant force = %.4f
N\n",result); break;
```

```
case 9:
```

```
density=positive("Solid density (kg/m^3): "); fluid_density=positive("Fluid
density (kg/m^3): ");
```

```
state=determine_buoyancy(density,fluid_density);
```

```
if(state==SINKS) printf("The solid SINKS.\n"); else if(state==FLOATS)
printf("The solid FLOATS.\n"); else printf("The solid remains
SUSPENDED.\n"); break;
```

```
case 10: printf("Program terminated.\n"); break;
```

```
default: printf("Invalid menu choice. Enter 1 to 10.\n");
```

```
}
```

```
} while (choice != 10);
```

```
return 0;
```

```
}
```

5. Compilation and Execution

Save all five files in the same directory. On Kali Linux, open a terminal in that directory and compile with

GCC:

```
$ gcc -std=c11 -Wall -Wextra -pedantic main.c geometry.c physics.c  
math_utils.c -o solid_project
```

main.c: In function 'main':

main.c:68:13: warning: this 'else' clause does not guard... [-Wmisleading-indentation]

```
68 | else if (result < 0) printf("Invalid input.\n"); else printf("Pressure = %.4f  
Pa\n",result); break;
```

```
| ^~~~
```

main.c:68:106: note: ...this statement, but the latter is misleadingly indented as if it were guarded by the 'else'

```
68 | else if (result < 0) printf("Invalid input.\n"); else printf("Pressure = %.4f  
Pa\n",result); break;
```

```
| ^~~~~
```

Run the executable using:

```
$ ./solid_project
```

The source supplied in this solution was compiled with GCC using -Wall -Wextra -pedantic; compilation completed without errors or warnings.

6. Sample Execution / Testing

The following is an actual scripted execution of the compiled program, covering geometry and all three

buoyancy states.

```
=== SOLID GEOMETRY AND PHYSICS MENU ===
```

1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit

Choice: Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): Radius (m):

Surface area = 50.2656 m^2

=== SOLID GEOMETRY AND PHYSICS MENU ===

1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit

Choice: Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): Radius (m):

Height (m): Volume = 62.8320 m^3

=== SOLID GEOMETRY AND PHYSICS MENU ===

1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit

Choice: Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): Length (m):
Breadth (m): Base area = 6.0000 m^2

=== SOLID GEOMETRY AND PHYSICS MENU ===

1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit

Choice: Solid density (kg/m^3): Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): Invalid solid/dimensions.

=== SOLID GEOMETRY AND PHYSICS MENU ===

1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit

Choice: Invalid menu choice. Enter 1 to 10.

=== SOLID GEOMETRY AND PHYSICS MENU ===

1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit

Choice: Solid density (kg/m^3): Fluid density (kg/m^3): The solid FLOATS.

=== SOLID GEOMETRY AND PHYSICS MENU ===

1. Surface area

2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit

Choice: Solid density (kg/m^3): Fluid density (kg/m^3): The solid remains
SUSPENDED.

=== SOLID GEOMETRY AND PHYSICS MENU ===

1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit

Choice: Program terminated.

7. Expected Results for Key Calculations

Case	Calculation	Expected
Sphere area $r=2$	$4 \times 3.1416 \times 2^2$	50.2656 m ²
Cylinder volume $r=2, h=5$	$3.1416 \times 2^2 \times 5$	62.8320 m ³
Cone area $r=3, h=4$	$\text{sqrt}(9+16)=5;$ $3.1416 \times 3 \times (3+5)$	75.3984 m ²
Cuboid volume $2 \times 3 \times 4$	$2 \times 3 \times 4$	24 m ³
Cylinder pressure $r=1, h=2, \rho=500$	$\rho \times V \times g / A = \rho \times h \times g$	9800 Pa
Cuboid buoyant force $1 \times 2 \times 3, \rho_f=1000$	$1000 \times 6 \times 9.8$	58800 N

8. Conclusion

The modular C project implements the required geometry and physics operations using decision-making and iterative constructs. Input validation prevents invalid dimensions, densities and choices from being used. The Newton-Raphson method provides the cone square-root calculation without math.h. The program separates menu handling, geometry, physics and mathematical utility functions into independent modules, making the solution easier to test, understand and maintain.

Compilation and Test Outputs

```
Session Actions Edit View Help
(kali[kali])-[-]
$ gcc -std=c11 -Wall -Wextra -pedantic main.c geometry.c physics.c math_utils.c -o solid_project
Compilation successful. No errors.
$ ./solid_project
=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 10
Program terminated.
```

Test Case1

```
Session Actions Edit View Help

(kali[kali])-[-]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 1
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 1
Radius (m): 2
Surface area = 50.2656 m^2
```

Test Case 2

```
Session Actions Edit View Help

(kali[kali])-[-]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 2
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 2
Radius (m): 2
Height (m): 5
Volume = 62.8320 m^3
```

Test Case 3

```
Session Actions Edit View Help

(kali[kali])-[-]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 1
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 3
Radius (m): 3
Height (m): 4
Surface area = 75.3984 m^2
```

Test Case 4

```
Session Actions Edit View Help

(kali[kali])-[~]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 2
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 4
Length (m): 2
Breadth (m): 3
Height (m): 4
Volume = 24.0000 m^3
```

Test Case 5

```
Session Actions Edit View Help

(kali[kali])-[~]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 3
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 2
Radius (m): 2
Base area = 12.5664 m^2
```

Test Case 6

```
Session Actions Edit View Help

(kali[kali])-[~]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 3
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 1
Base area is not defined for a sphere.
```

Test Case 7

```
Session  Actions  Edit  View  Help

(kali[kali])-[~]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 1
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 1
Radius (m): 0
Value must be greater than zero. Try again.
Radius (m): 1
Surface area = 12.5664 m^2
```

Test Case 8

```
Session  Actions  Edit  View  Help

(kali[kali])-[~]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 2
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 2
Radius (m): 1
Height (m): -2
Value must be greater than zero. Try again.
Height (m): 2
Volume = 6.2832 m^3
```

Test Case 9

```
Session  Actions  Edit  View  Help

(kali[kali])-[~]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 6
Elevation (m, >= 0): 0
Enter solid density (kg/m^3): 1000
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 1
Enter radius (m): 1
Potential energy = 0.0000 J
```

Test Case 10

```
Session  Actions  Edit  View  Help

(kali[kali])-[-]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 4
Solid density (kg/m^3): 1000
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 1
Enter radius (m): 1
Mass = 4188.8000 kg
```

Test Case 11

```
Session  Actions  Edit  View  Help

(kali[kali])-[-]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 7
Solid density (kg/m^3): 500
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 2
Enter radius and height (m): 1 2
Pressure = 9800.0000 Pa
```

Test Case 12

```
Session  Actions  Edit  View  Help

(kali[kali])-[-]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 8
Fluid density (kg/m^3): 1000
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 4
Enter length, breadth and height (m): 1 2 3
Buoyant force = 58800.0000 N
```

Test Case 13

```
Session  Actions  Edit  View  Help

(kali[kali])-[~]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 9
Solid density (kg/m^3): 7800
Fluid density (kg/m^3): 1000
The solid SINKS.
```

Test Case 14

```
Session  Actions  Edit  View  Help

(kali[kali])-[~]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 9
Solid density (kg/m^3): 900
Fluid density (kg/m^3): 1000
The solid FLOATS.
```

Test Case 15

```
Session  Actions  Edit  View  Help

(kali[kali])-[~]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 9
Solid density (kg/m^3): 1000
Fluid density (kg/m^3): 1000
The solid remains SUSPENDED.
```

Test Case 16

```
Session  Actions  Edit  View  Help

(kali[kali])-[-]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 1
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 5
Invalid solid choice. Try again.
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 2
Radius (m): 1
Height (m): 2
Surface area = 18.8496 m^2
```

Test Case 17

```
Session  Actions  Edit  View  Help

(kali[kali])-[-]
$ ./solid_project

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 0
Invalid menu choice. Enter 1 to 10.

=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 10
Program terminated.
```

Test Case 18

```
Session Actions Edit View Help

(kali[kali])-[~]
$ ./solid_project
=== SOLID GEOMETRY AND PHYSICS MENU ===
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Choice: 4
Solid density (kg/m^3): 0
Value must be greater than zero. Try again.
Solid density (kg/m^3): 1000
Select solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): 1
Enter radius (m): 1
Mass = 4188.8000 kg
```

Test Case 19

```
Session Actions Edit View Help

(kali[kali])-[~]
$ ./solid_project
$ gcc -std=c11 square_root_test.c math_utils.c -o sqrt_test
$ ./sqrt_test
Enter number: 0
calculate_square_root(0.0000) = 0.0000
```

Test Case 20

```
Session Actions Edit View Help

(kali[kali])-[~]
$ ./solid_project
$ gcc -std=c11 square_root_test.c math_utils.c -o sqrt_test
$ ./sqrt_test
Enter number: -4
calculate_square_root(-4.0000) = -1.0000
```